

KPC FlowGuard

Autonomous Operational Control Plane for Petroleum Depot Turnaround

Inuka Hackathon #3 — KPC Morendat Hub | Team Submission, Part D

Repo: github.com/Michael-Munga/FlowGuard

1. The Problem

KPC depots (Nairobi PS10, Mombasa KOT, Nakuru, Eldoret, Kisumu) process hundreds of petroleum uplift orders daily. Delays compound from arrival clustering, product-incompatible bay assignment, equipment faults, and manual customs/manifest validation. Demurrage accrues at KES 20 per tanker per minute — a 30-minute delay across 6 tankers alone creates KES 3,600 in exposure per hour, scaling materially across five depots and hundreds of daily orders.

2. The Solution

FlowGuard is an operational control plane — not a dashboard — following an 8-stage Sense-to-Audit loop: Sense (ingest telemetry) → Predict (dual XGBoost models forecast arrival & turnaround) → Diagnose (0–100 explainable risk score) → Optimize (OR-Tools CP-SAT bay allocation) → Decide (3-tier policy: Auto-Executable / Approval-Required / Blocked) → Execute (simulated dispatch adapter) → Verify (compare against counterfactual baseline) → Log (SHA-256 sealed audit chain).

Built: FastAPI + SQLAlchemy + PostgreSQL 15 backend (22 tables, 89,338 canonical records after Great Expectations validation), Next.js 16/React 19 frontend with six role-based experiences, and 94 automated tests across 14 test files.

3. Verified Results (Synthetic Evaluation)

All figures below are measured against the project's own synthetic 97-day dataset (11,031 orders, 5 depots, 7 OMCs) — not live KPC data. The system is explicitly not connected to any real KPC SCADA, gantry, or weighbridge hardware; every execution is simulated.

Metric	Baseline	FlowGuard
Turnaround prediction error (MAE)	22.98 min (static target)	10.33 min — 55% error reduction
Turnaround predictions within 10 min	23.8%	55.1%
Demo decision DEC-0142 (Nairobi, critical-risk scenario)	—	-34 min turnaround, 89.5% recovery attainment
Cumulative realized savings (synthetic audit trail)	—	KES 43M+
Autonomy adoption ratio (synthetic)	—	~60%
Optimizer API round-trip	—	~100–200 ms

4. Why FlowGuard Wins

- Predictive, not reactive — forecasts congestion before trucks are already blocking the gate, instead of triaging after the fact.
- Bounded autonomy — routine allocations execute automatically within strict policy limits; high-risk or high-impact cases (>6 orders, risk ≥85, confidence <75%) are routed to human approval, never auto-dispatched.
- Fully auditable — every decision is sealed in an append-only SHA-256 chain, retrievable via API for dispute resolution and compliance.
- Honest about its own limits — the system explicitly reports simulated execution status and synthetic-data caveats on every relevant output, rather than overstating readiness.

5. Current Status & What's Left (self-assessed against Stage 3 rubric)

Rubric Criterion	Status	Evidence
Deployment, Quality & Resilience	Partial	Postgres via Docker Compose, FastAPI local, health/readiness probes built. No CI/CD or public deployment yet.
Quantified ROI & Business Case	Partial	Counterfactual estimates + KES 43M+ synthetic realized savings computed; all figures clearly marked as synthetic, not live KPC results.
Integration & Usability	Partial	Six dashboards built; core API-backed data

		domains wired; some widget-level data still synthetic in API mode.
Documentation & Handover	Complete	Full README plus API contract, database design, model cards, ETL and policy docs.
Pitch & Executive Storytelling	Partial	Executive Control Plane view built with ROI waterfall; formal pitch deck/recording still needed.

Remaining for full submission: the live pitch recording from the hackathon, and (if pursued further) CI/CD, live deployment, and full widget-level API wiring.